

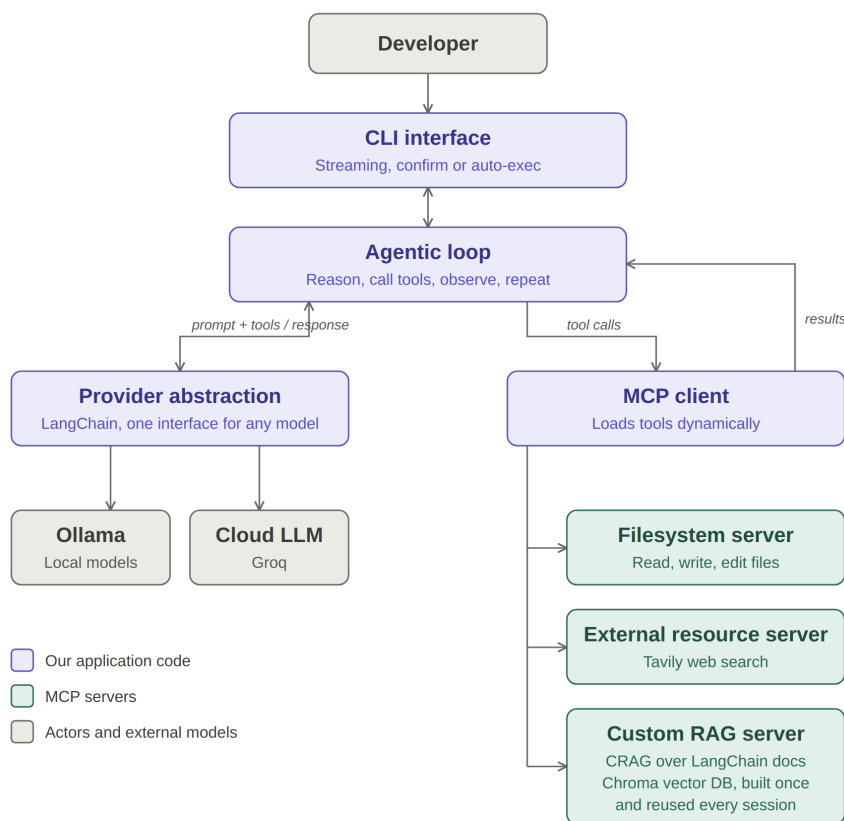
Forge: Project Plan

What we're building

Forge is a command line AI coding assistant, similar to Claude Code or OpenCode. You give it a task in plain English, and it figures out which tools to use, reads and edits files, searches the web or our docs when it needs to, and keeps going until the task is done.

The name is a working name and may change. Final submission is due Oct 15, with a check-in on Oct 8.

Initial architecture



Flow: the developer enters a task in the CLI. The agentic loop sends it to an LLM through the provider abstraction. The model chooses tools, which run through the MCP client against the three servers. Results go back to the loop, which repeats until the task is complete.

1. The developer types a task into the CLI.
2. The agentic loop sends the task, the conversation history, and the list of available tools to the LLM through the provider abstraction. This works with Ollama (local) or Groq (cloud).
3. The LLM either answers or picks one or more tools to call.
4. In confirm mode, the developer approves each tool call before it runs. In auto mode, it runs right away.

5. The MCP client sends each call to the right server: filesystem for files, Tavily for web search, or our custom RAG server for LangChain docs.
6. The results go back to the loop, which passes them to the LLM and repeats until the task is done.

Planned tech stack

Part	Choice	Why
Language	Python 3.11+	Everyone on the team knows it
LLM providers	Ollama (local), Groq (cloud)	Groq has a free tier, so no one has to pay
Provider abstraction	LangChain chat models	One interface for both providers
MCP client	MCP Python SDK	Official SDK, supports stdio servers
Filesystem server	@modelcontextprotocol/server-f ilesystem	Required by the assignment
External server	Tavily MCP	Web search, free tier
Custom RAG server	FastMCP, ChromaDB, sentence-transformers	Simple to build, runs locally
RAG docs	LangChain documentation	Useful for the assistant itself
Advanced RAG technique	Corrective RAG (CRAG)	Grades retrieved chunks and retries with a better query when results are weak
CLI	rich, prompt_toolkit	Streaming, panels, spinners, input history

Team roles

Member	Main responsibility
Varshita	Custom RAG MCP server: ingest script, Chroma DB, CRAG retrieval, server tools
Abi	Agentic loop, provider abstraction (Ollama and Groq), confirm and auto modes
Aadi	MCP client, filesystem server, Tavily server, dynamic tool loading, CLI display

Shared by all three: state and sequence diagrams, LLM comparison, report, README, demo video, and testing. Roles can shift if one part takes longer than expected.

Timeline

Week 1 (Oct 1 to Oct 7): Planning and core pieces

- Oct 1: Commit the planning docs and initial architecture diagram (all)
- Oct 2 to 3: Repo setup, requirements.txt, and .env.example (all)
- Oct 2 to 7: Agentic loop working with Ollama and Groq (Abi)
- Oct 3 to 6: MCP client connected to the filesystem and Tavily servers (Aadi)
- Oct 3 to 7: RAG ingest script that loads, chunks, embeds, and stores docs in Chroma (Varshita)
- Oct 7: Practice run for the check-in (all)

Week 2 (Oct 8 to Oct 15): Finish, evaluate, submit

- Oct 8: Check-in. Demo the filesystem and Tavily servers working through the agent
- Oct 8 to 10: RAG server with CRAG, connected to the MCP client (Varshita)
- Oct 8 to 10: Confirm and auto modes, plus shell commands (Abi)
- Oct 9 to 11: CLI polish and testing with real coding tasks (Aadi)
- Oct 11 to 12: Compare two LLMs on the same task, and basic RAG vs CRAG (all)
- Oct 12 to 13: State and sequence diagrams (all)
- Oct 13 to 14: Report, README, and demo video (all)
- Oct 15: Final review and submit

Extras like session saving only happen if we finish early.

Repo structure and how we'll work

```
forge/
├── planning/      # this plan and the architecture diagram
├── forge/
│   ├── cli.py     # REPL and display
│   ├── agent.py   # agentic loop
│   ├── providers.py # Ollama and Groq setup
│   ├── mcp_client.py # connects to servers and loads tools
│   └── config.py
├── rag_server/
│   ├── ingest.py  # one-time build of the vector DB
│   ├── server.py  # FastMCP server
│   └── crag.py    # corrective RAG logic
├── docs/          # report and diagrams
├── requirements.txt
├── .env.example
└── README.md
```

- Each person works on their own branch and opens a pull request into main
- At least one teammate reviews a PR before it gets merged
- We check in as a group twice a week to share progress and blockers
- API keys stay in .env, which is in .gitignore